PDF 필기 자료 (코넬 노트)

2026년 1학기 · Games, CSP/Factor Graph, Markov/Bayesian Net, Logic · 총 35쪽

인공지능 필기본 — 코넬

Games · CSP/Factor Graph · Markov/Bayesian Net · Logic (강의 09-18) · 회독용

Ch 09 Games 1 — Minimax · Evaluation · Alpha-Beta

2-player zero-sum game tree search · depth-limited eval · alpha-beta pruning

Game Tree & Minimax p.5-20

Q. Minimax V_max/V_min 점화식은?

Q. 상대를 어떻게 가정하나?

Q. terminal·utility의 정의는?

Two-player zero-sum game — 정의 — agent 효용이 올라가면 opp 효용이 같은 크기로 내려가는 게임.

체스 예: Utility= +∞ 백 승 / 0 무승부 / −∞ 흑 승

- **s_start** : 시작 상태
- `Actions(s)` : s에서 가능한 행동 집합
- `Succ(s,a)` : (s,a) → 다음 상태
- `IsEnd(s)` : 종료 여부
- `Utility(s)` : 종료 상태에서만 정의되는 agent 효용
- `Player(s)` ∈ {agent, opp} : s를 제어하는 플레이어

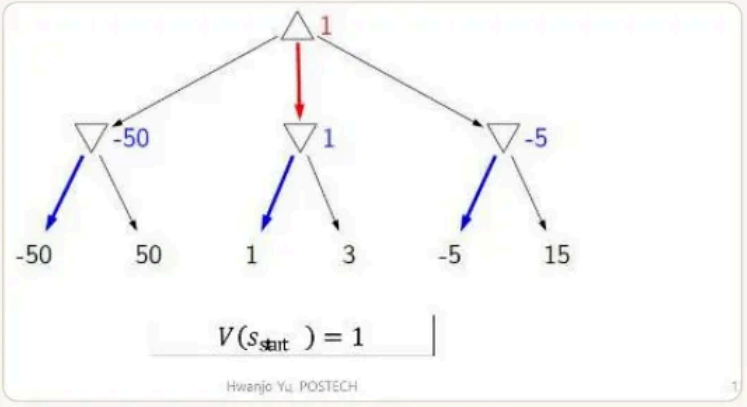
Game Tree — 핵심 아이디어 — 각 노드 = 플레이어의 *결정점*. root-to-leaf 경로 = 게임 가능 결과 하나.

- △ max-node(agent 차례) : 효용 최대화
- ▽ min-node(opp 차례) : 효용 최소화
- ◇ chance-node : 확률 가중 기댓값

Minimax 점화식 — opp가 최악(agent에게 불리)을 선택한다고 가정. MDPs에 대응 없음.

opp 최악 가정 = minimax

- IsEnd(s): V(s) = Utility(s)
- Player(s)=agent: V(s) = max over a∈Actions(s) of V(Succ(s,a))
- Player(s)=opp: V(s) = min over a∈Actions(s) of V(Succ(s,a))



Minimax example — V(s_start)=1 (p.15)

정책 추출

- π_agent(s) = argmax_a V(Succ(s,a))
- π_opp(s) = argmin_a V(Succ(s,a))

Minimax 두 성질

- **Property 1 — Best against minimax opponent**: V(π_agent, π_opp)(s) ≥ V(π, π_opp)(s) for all π → minimax 상대에게는 minimax 정책이 최선
- **Property 2 — Lower bound against any opponent**: V(π_agent, π_opp)(s) ≤ V(π_agent, π)(s) for all π → 어떤 상대든 minimax 정책이면 V(π_agent,π_opp)가 하한 보장
- **Non-property 3**: minimax 정책이 모든 상대에게 최적은 아님 — opp가 최악이 아닌 경우 손해 가능

✎ 핵심 minimax = 최악 상대 가정 → 보수적 보장. opp가 실수하면 기대보다 더 잘함. 'non-property 3'이 핵심 — 무조건 최적이 아님.

Expectiminimax — 확장 — Players에 coin(chance) 추가 시 점화식에 chance-node 추가.

- Player(s)=coin: V(s) = Σ_a p(s,a)·V(Succ(s,a)) (확률 가중합)
- Primitives: max, min, chance-node를 *모델에 따라 조합*하면 어떤 게임도 표현 가능

Minimax 복잡도 — 분기 인수 b, 깊이 d(ply 단위). 공간 O(bd), 시간 O(b^d).

- 체스: b≈35, d≈50 → 완전 탐색 불가(천문학적)
- 두 가지 가속: (1) Evaluation function — 근사값, (2) Alpha-beta pruning — 정확값

Evaluation Functions p.27-36

Q. depth-limited search 가 왜 필요?

Q. evaluation function 의 역할은?

Q. 근사일 뿐인 이유는?

Depth-limited Search — 최대 깊이 d_max에서 탐색 멈추고 Eval(s) 호출. 시간: O(b^d_max)

- V(s, d) = Eval(s) if d=0 or IsEnd(s)
- max/min over successors of V(Succ, d−1) otherwise
- 관례: 마지막 플레이어 차례에 depth 1 차감

Eval(s) — 정의 & 성질 — V(s)의 (약한) *추정치*. 검색 문제의 FutureCost(s)에 대응.

- 체스 예: Eval(s) = material + mobility + king-safety + center-control
- material = 10(K−K') + 9(Q−Q') + 5(R−R') + 3(B−B') + 3(N−N') + 1(P−P')

선형 함수 근사 — Parameterized Eval — Eval(s; w) = w^T · f(s). w∈ℝⁿ 가중치, f(s) = 특징 벡터.

- f_i = (내 말 개수) − (상대 말 개수) 등 도메인 특징
- w를 학습으로 결정 → supervised / self-play 강화학습

참 가치함수 근사 — 두 방법 — 최적 정책 π*, π'_opp를 알면 Eval(s)=V(s) (intractable). 두 근사:

- ① Stochastic policies: π, π'를 확률 정책으로 교체 → 단순화되나 여전히 계산량 있음
- ② Monte Carlo: K번 random rollout → Eval(s) ≈ (1/K)Σ util_k

💡 TIP Monte Carlo Eval: K=10 예시 → (1+3+50+50+50−50−50+50+15−5)/10 = 11.4. AlphaGo = MCTS(탐색) + policy net(스토캐스틱 π) + value net(Eval).

Alpha-Beta Pruning p.38-44

Q. α·β의 의미는?

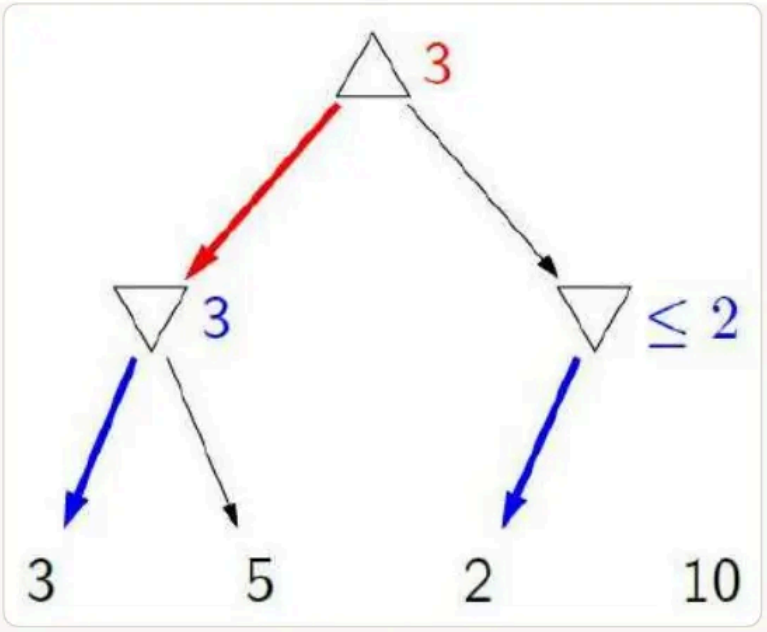
Q. 가지치기 조건(β≤α)은?

Q. 최선 정렬 시 복잡도는?

Q. 답이 보존되는 이유는?

프루닝 원리 — Branch and Bound — 구간 [α, β]가 겹치지 않으면 추가 계산 없이 선택 가능.

- 예: A=[3,5], B=[5,100] → max 선택은 B로 확정, A 추가 탐색 불필요
- Alpha-beta = BnB를 minimax 트리에 특화한 것



β≤2이므로 우측 서브트리 불필요 — max(3,≤2)=3 (p.39)

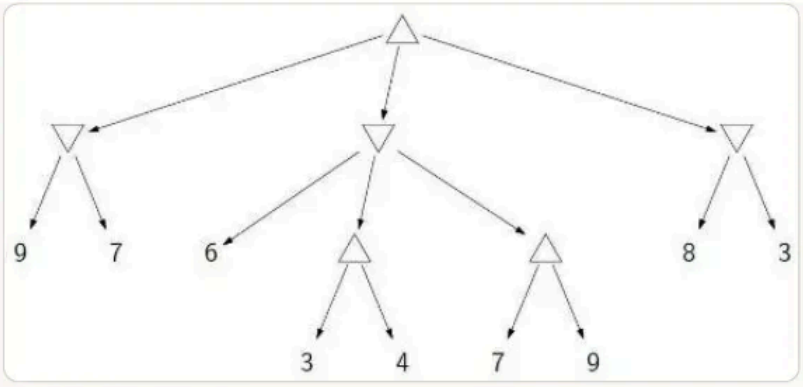
α·β 의미

- α = max-node의 하한 — 지금까지 본 자식 중 최댓값
- β = min-node의 상한 — 지금까지 본 자식 중 최솟값
- Optimal path의 모든 노드 값은 동일 (optimal path property)
- 조상 중 max-node의 α 최댓값 = A, min-node의 β 최솟값 = B로 유지
- Prune 조건: β ≤ α (상한이 하한 이하 → 구간 교집합 없음)

Alpha-Beta Pruning

```
function alphabeta(node, depth, α, β, maximizingPlayer)
  if depth = 0 or node is terminal
    return Eval(node)
  if maximizingPlayer
    v := −∞
    for each child of node
      v := max(v, alphabeta(child, depth−1, α, β, FALSE))
      α := max(α, v)
      if β ≤ α   ← β-cutoff: min-ancestor가 이미 β≤α 확인
        break
    return v
  else
    v := +∞
    for each child of node
      v := min(v, alphabeta(child, depth−1, α, β, TRUE))
      β := min(β, v)
      if β ≤ α   ← α-cutoff: max-ancestor가 이미 α≥β 확인
        break
    return v
호출: alphabeta(root, depth, −∞, +∞, TRUE)
```

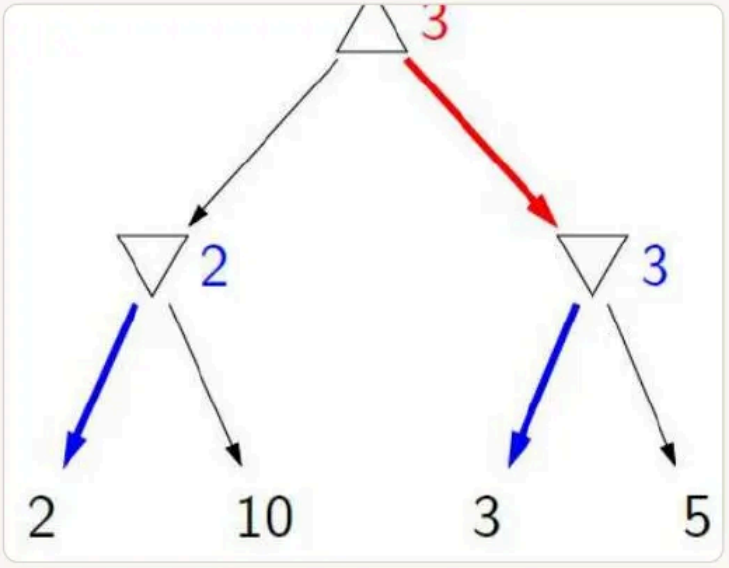
β≤α = 상한이 하한 이하 → 이 서브트리가 조상 선택에 영향 못 줌 → prune.



Alpha-beta pruning example tree (p.41)

Move Ordering — 복잡도 — Pruning 효과는 행동 탐색 순서에 의존.

- 최악 순서: O(b^d) — pruning 없음
- 최선 순서: O(b^(d/2)) — 홀수 깊이마다 1개만 탐색, effective branching ≈ √b
- 랜덤 순서: O(b^(3d/4))
- 실전: `Eval(s)` 로 max-node는 내림차순, min-node는 오름차순 정렬



순서 바뀌면 5-node pruning 불가 (p.43)

	Worst ordering	Best ordering	Random ordering
시간복잡도	O(b^d)	O(b^(d/2))	O(b^(3d/4))
chess 효과	b=35,d=50 불가	b=6,d=50 가능권	중간

move ordering 품질이 pruning 효율을 O(b^d/2) vs O(b^d) 차이만큼 바꿈

★ 필수암기 α-cutoff: β≤α → max-node에서 break. β-cutoff: β≤α → min-node에서 break. 최선 정렬 시 effective 분기 √b → 같은 시간에 2배 깊이 탐색. minimax 값 변경 없이 동일 결과(exact, not approximate).

⚠ 함정 Alpha-beta pruning은 정확한 답을 내지만 평균 pruning 비율은 move ordering에 의존. Eval 호출 비용 vs pruning 이득 trade-off 고려.

요약 Minimax = max/min 점화식으로 **최악 상대 가정** 하 V(s) 계산 · Eval+depth-limit으로 근사 · Alpha-Beta는 β≤α 시 서브트리 prune(답 불변) · 최선 정렬 시 **O(b^(d/2))**로 깊이 2배 탐색.

Ch 10 Games 2 — TD Learning · Simultaneous · Non-zero-sum

강화학습 기반 게임 평가함수 학습 + 동시수행 게임 minimax + Nash 균형

TD Learning p.1-15

Q. TD error(prediction–target)는?

Q. w 갱신식은? on-policy인 이유?

Q. Q-learning이 off-policy인 까닭?

평가함수 파라미터화 — 보드 상태 `s` 를 feature vector `ϕ(s)` 로 표현, **가중치 w** 를 학습. `V(s;w) = w·ϕ(s)` (선형 함수일 때).

- Backgammon 예: # of pieces in each column, fraction removed 등 **수치 feature** 사용
- Self-play로 데이터 생성 → π_agent(max), π_opp(min) 기반 episode

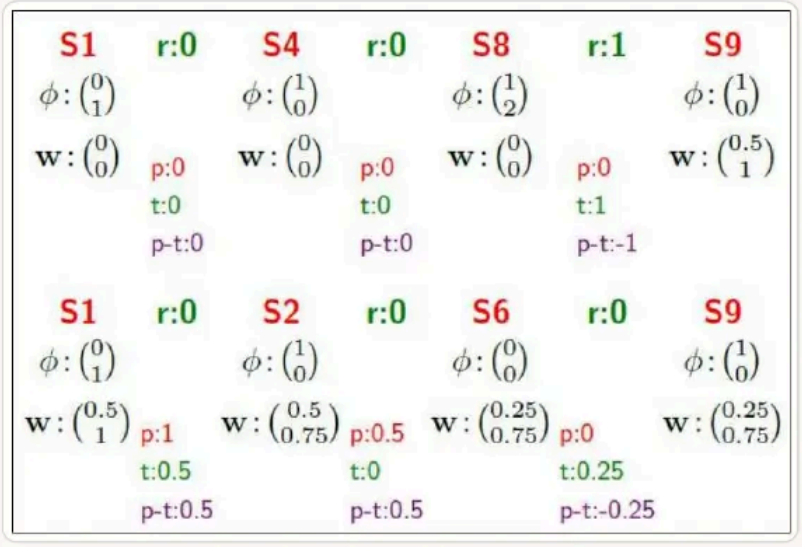
Episode & Prediction-Target 구조 — 에피소드: $s_0;\ a_1,r_1,s_1;\ a_2,r_2,s_2;\ \ldots;\ a_n,r_n,s_n$

- Prediction = `V(s;w)` — 현재 가중치의 예측
- Target = `r + γ·V(s´;w)` — 다음 상태 + 즉시보상으로 만든 목표치
- 목적함수: ½(prediction − target)² 를 최소화

TD Learning (on-policy)

```
On each transition (s, a, r, s'):
  w ← w − η · [V_π(s;w) − (r + γ·V_π(s';w))] · ∇_w V_π(s;w)
For linear functions:
  V(s;w)      = w · ϕ(s)      ← 선형 평가함수
  ∇_w V(s;w) = ϕ(s)           ← gradient = feature vector
```

η=학습률, γ=discount. prediction–target = TD error. ∇=feature vector(선형).



TD learning 추적 예제 — η=0.5, γ=1, 종단 보상만 (p.9)

Q-Learning (off-policy TD)

```
On each transition (s, a, r, s'):
  Q(s,a;w) ← Q(s,a;w) − η · [Q(s,a;w) − (r + γ·max_{a'} Q(s',a';w))] · ∇Q(s,a;w)
```

off-policy: 다음값이 optimal policy 기반(max). MDP transition 불필요.

TD vs Q-Learning 비교

- TD learning: `V(s;w)` 연산 · on-policy(탐색 정책 기반) · Succ(s,a) 필요
- Q-learning: `Q(s,a;w)` 연산 · off-policy(최적 정책 추정) · transition model 불필요

✎ **핵심** TD error = prediction − target. 선형일 때 gradient = `ϕ(s)` 자체. Q-learning은 max a' 를 써서 off-policy, TD는 π 기반이라 on-policy.